



اُونِيْوَرْسِيْتِيْ تِيْكْنُوْلُوْجِيْ مَّارَا  
UNIVERSITI  
TEKNOLOGI  
MARA

**UNIVERSITI TEKNOLOGI MARA**

**FACULTY OF COMPUTING, INFORMATICS AND MATHEMATICS**

Degree of Information Science

CDIM262

IMS566

ADVANCED WEB DESIGN DEVELOPMENT AND CONTENT MANAGEMENT

Grouping Assignment

Prepared by:

**MUHAMMAD TAUFIQ DANIEL BIN ABDULLAH**

**(2024917109)**

29 January 26

# STUDENT PORTAL

Prepared by:

Muhammad Taufiq Daniel Bin Abdullah

(2024917109)

Faculty of Computing, Informatics and Mathematics

Universiti Teknologi Mara

Selangor

## Acknowledgement

First and foremost, I would like to express my deepest gratitude to my lecturer, Sir Asyraf, for their invaluable guidance, patience, and constructive feedback throughout the preparation of this (IMS566) proposal. Their insights were instrumental in shaping the scope and direction of the project.

A special thanks goes to my family for their unwavering support, understanding, and encouragement during this semester. Their belief in my potential has been my greatest motivation.

Finally, I would like to thank my friends and fellow classmates for their willingness to exchange ideas, provide moral support, and offer suggestions during the brainstorming phases of this assignment.

The completion of this proposal report would not have been possible without the support of everyone mentioned above.

| TABLE OF CONTENT                      | Page Number |
|---------------------------------------|-------------|
| 1.0 Introduction                      | 5           |
| 2.0 Github Repository Link            | 5           |
| 3.0 Entity-Relationship Diagram (ERD) | 6           |
| 4.0 System Requirements               | 7           |
| 5.0 User Interface (UI)               | 8           |
| 6.0 Features and Functionalities      | 8           |
| 7.0 Workflow of Form (CRUD)           | 12          |
| 8.0 Team Roles & Contribution         | 12          |
| 9.0 Contact and Information           | 13          |
| 10.0 Conclusion and Reflection        | 13          |

## **Student Portal - System Report & User Manual**

### **1.0 Introduction**

In contemporary learning institutions, effective student data management is a key determinant of success. Conventional approaches to data management, such as manual filing and the use of simple spreadsheets, are often susceptible to human error, data duplication, and security breaches.

The Student Management Portal is a solution to these problems. This project is a full-fledged web-based application that aims to computerize the management of student data. The application is developed using the CakePHP 5 framework and utilizes the Model-View-Controller (MVC) design pattern to ensure the modularity, scalability, and security of the code. The main aim of this application is to enable administrators to have a central place to create, read, update, and delete student data while upholding the highest standards of data integrity and authentication.

### **2.0 GitHub Repository Link**

The complete source code for this project, including version history and configuration files, is hosted on GitHub. This repository demonstrates the development progression and adheres to industry-standard version control practices.

- Repository URL: <https://github.com/danielozy/student-portal>
- Developer: Taufiq Daniel

### 3.0 Entity-Relationship Diagram (ERD)

The database schema was designed to optimize data storage and retrieval. The system utilizes a relational database structure consisting of two primary entities: Users (for administration) and Students (for record storage).

| USERS        |          |    |                |  | STUDENTS     |            |        |                |  |
|--------------|----------|----|----------------|--|--------------|------------|--------|----------------|--|
| INT          | id       | PK | Auto Increment |  | INT          | id         | PK     | Auto Increment |  |
| VARCHAR(255) | email    |    |                |  | VARCHAR(50)  | matric_no  | Unique |                |  |
| VARCHAR(255) | password |    | Hashed         |  | VARCHAR(100) | first_name |        |                |  |
| DATETIME     | created  |    |                |  | VARCHAR(100) | last_name  |        |                |  |
| DATETIME     | modified |    |                |  | VARCHAR(255) | email      |        |                |  |
|              |          |    |                |  | VARCHAR(100) | course     |        |                |  |
|              |          |    |                |  | VARCHAR(50)  | level      |        |                |  |
|              |          |    |                |  | DATETIME     | created    |        |                |  |
|              |          |    |                |  | DATETIME     | modified   |        |                |  |

ERD Diagram

a) The Users Entity (Admin Table) Designed for security and access control.

- id (INT, Primary Key): A unique identifier for the system admin.
- email (VARCHAR): The unique login credential for the administrator.
- password (VARCHAR): Stores the bcrypt-hashed password string for security.
- created/modified (DATETIME): Timestamps for auditing account activity.

b) The Students Entity (Data Table) Designed to store comprehensive academic profiles.

- id (INT, Primary Key): The internal system reference ID.
- matric\_no (VARCHAR, Unique): The distinct student identification number; the system prevents duplicates here.
- first\_name / last\_name (VARCHAR): Stores the student's personal identity.

- email (VARCHAR): Contact information.
- course / level (VARCHAR): Academic details regarding their current study path.
- created/modified (DATETIME): Timestamps to track when records are entered or updated.

#### 4.0 System Requirements

To ensure optimal performance and compatibility, the Student Management Portal requires a specific server environment. The application was developed and tested using the following configuration:

- Server Stack: Laragon (WAMP - Windows, Apache, MySQL, PHP).
- Server-Side Language: PHP Version 8.1 or higher (Required for CakePHP 5).
- Web Framework: CakePHP 5.x (Utilizing strict typing and MVC structure).
- Database Management: MySQL (Relational Database Management System).
- Dependency Management: Composer (For handling PHP libraries and vendor packages).
- Client-Side Interface: HTML5, CSS3, and Bootstrap 5 Framework.

## 5.0 User Interface (UI) Overview

The user interface was constructed with a "Mobile-First" philosophy using the Bootstrap 5 framework. This ensures that the application is fully responsive, adapting seamlessly to desktop monitors, tablets, and mobile screens.

- **Navigation Architecture:** The top navigation bar serves as the primary control center. It dynamically adapts based on the user's session state—hiding sensitive links (like "New Student") from guests and displaying them only after a successful login.
- **Visual Feedback System:** The application implements "Flash Messages." These are distinct notification bars (Green for success, Red for errors) that appear instantly after an action is performed, providing the user with immediate confirmation of their tasks.
- **Data Presentation:** Student records are displayed in a clean, striped table layout, improving readability and allowing administrators to scan large datasets quickly.

## 6.0 Features & Functionalities

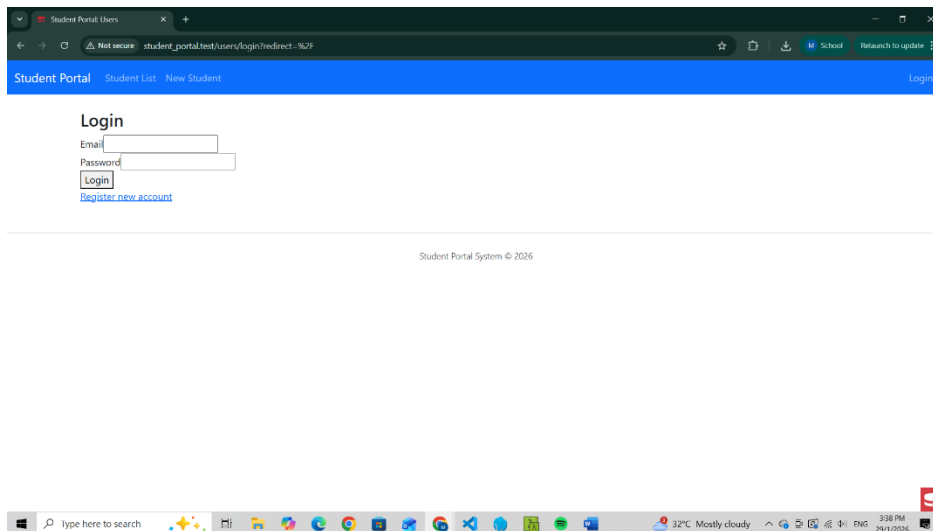
The system is packed with essential features designed for security and usability:

1. **Secure Administrator Authentication** Security is paramount. The system utilizes CakePHP's Authentication plugin. Passwords are never stored in plain text; instead, they are encrypted using bcrypt hashing algorithms. If a user attempts to access internal pages without logging in, the "Bouncer" middleware intercepts the request and redirects them to the login page.
2. **Student Record Management (CRUD)** The core functionality allows for the lifecycle management of student data. Administrators can register new students via a form, view a comprehensive list of all records, update details (such as correcting a misspelled name), and permanently delete obsolete records from the database.
3. **Data Integrity & Validation** To prevent "garbage data" from entering the system, strict validation rules are enforced at the Model layer.
  - **Uniqueness:** The system prevents two students from sharing the same Matriculation Number.
  - **Completeness:** It is impossible to save a record if the Name or Email fields are empty.



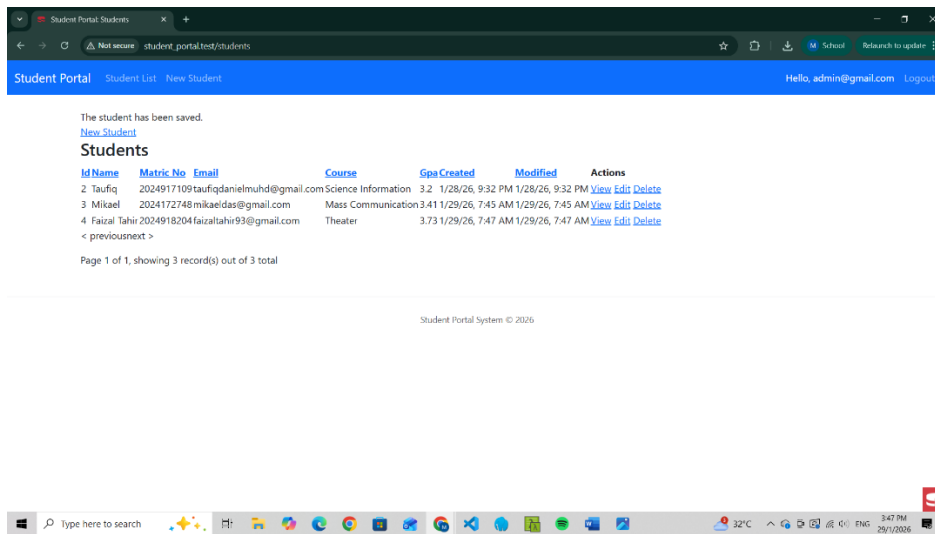
- Format: The system ensures that emails follow a valid format (e.g., user@domain.com).
4. Route Protection Using middleware logic, specific routes (URLs) are protected. A standard user cannot "guess" the URL to add a student; the server verifies their identity session before granting access to the controller actions.

### Authentication:



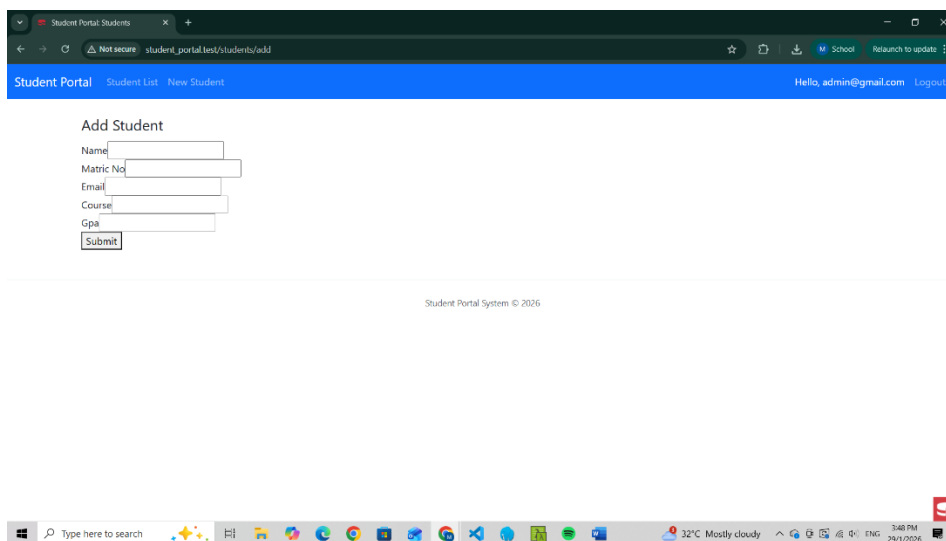
- Description: Secure entry point using hashed passwords. Prevents unauthorized access to student data.

## Student List (Read):



- Description: Displays student ID, Name, Matric No, and Course in a paginated table.

## Add Student (Create):



- Description: A validation-protected form to register new students.

## Edit Student (Update):

The screenshot shows a web browser window with the URL `student_portal.test/students/edit/2`. The page title is "Student Portal" and the user is logged in as "admin@gmail.com". The form is titled "Edit Student" and contains the following fields: Name (taufiq), Matric No (2024917109), Email (taufiqdanielmuhd@gmail.com), Course (Science Information), and Gpa (3.20). There are "Save Changes" and "Cancel" buttons. The footer of the page says "Student Portal System © 2026".

- Description: Allows admins to correct mistakes in student records.

## Validation Errors:

The screenshot shows a web browser window with the URL `student_portal.test/students/add`. The page title is "Student Portal" and the user is logged in as "admin@gmail.com". The form is titled "Add Student" and contains the following fields: Name, Matric No, Email, Course, and Gpa. A red error message "A student name is required" is displayed next to the Name field. There is a "Submit" button. The footer of the page says "Student Portal System © 2026".

- Description: The system prevents "Bad Data" (empty names or duplicate matric numbers) from entering the database.

## 7.0 Workflow of Form (The CRUD Cycle)

The application follows a strict Request-Response cycle typical of MVC architectures:

1. The Request: The user interacts with the View (e.g., clicking "Save Student").
2. Routing: The application directs this request to the specific Controller (e.g., StudentsController).
3. Processing: The Controller analyzes the data and sends it to the Model (StudentTable).
4. Validation: The Model checks the data against the defined rules (is the name empty? is the ID unique?).
5. Database Transaction: If valid, the data is committed to the MySQL database.
6. The Response: The Controller instructs the View to render a success message and redirects the user back to the main list.

## 8.0 Team Roles & Contributions

This project was executed by a dedicated Full Stack Developer.

- Developer Name: Taufiq Daniel
- Role: Lead Developer & System Architect
- Key Contributions:
  - Backend Engineering: Configured the Laragon environment and established the database connections.
  - Database Design: Structured the MySQL schema to ensure data normalization.
  - Logic Implementation: Coded the Controllers and Models within CakePHP to handle business logic.
  - Frontend Development: Designed the visual layout using Bootstrap to ensure a professional user experience.
  - Deployment Operations: Managed the Git version control system and successfully deployed the repository to GitHub.

## 9.0 Contact Information

For technical inquiries, bug reports, or system maintenance requests, please contact the development team:

- Lead Developer: Muhammad Taufiq Daniel Bin Abdullah
- GitHub Portfolio: <https://github.com/danielozy>
- Email: [taufiqdanielmuhd@gmail.com](mailto:taufiqdanielmuhd@gmail.com)

## 10.0 Conclusion & Reflection

The development of the Student Management Portal served as a comprehensive exercise in modern web engineering. By utilizing CakePHP 5, the project successfully demonstrated the power of rapid development frameworks in building secure, scalable applications.

One of the significant technical milestones achieved during this project was the implementation of the Authentication Layer. Ensuring that user sessions were managed securely required a deep understanding of middleware and password hashing protocols. Furthermore, enforcing strict validation rules highlighted the importance of data sanitation in preventing database errors.

In conclusion, the system meets all functional requirements: it is secure, user-friendly, and technically robust. It stands as a completed foundation, ready for future enhancements such as grade tracking, attendance monitoring, or reporting modules.